

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №1
«Методы сортировки»
Вариант 8

Выполнил:
Коцин А. О.

Москва
2026

Содержание

Постановка задачи	2
Результаты экспериментов	3
Экспериментальные данные	3
Heap Sort	3
Shell Sort	3
Анализ результатов	4
Структура программы и спецификация функций	4
sort.h	4
sort.c	5
main.c	5
Отладка программы, тестирование функций	6
Анализ допущенных ошибок	7
Список цитируемой литературы	9

Постановка задачи

В данной работе требуется реализовать и экспериментально сравнить два алгоритма сортировки массивов целых чисел: пирамидальную сортировку (Heap Sort) и сортировку Шелла (Shell Sort). Оба алгоритма сортируют массив **по убыванию**.

В ходе экспериментов необходимо:

- реализовать оба метода сортировки с подсчётом количества операций сравнения и перемещения элементов;
- провести тестирование на массивах различных размеров: $n = 10, 100, 1000, 10000$;
- использовать четыре типа входных данных для оценки поведения алгоритмов при различных степенях упорядоченности: отсортированные данные, данные в обратном порядке, два варианта случайных данных;
- провести анализ полученных результатов и сравнить эффективность алгоритмов.

Сравнение производится по количеству операций сравнения и обмена элементов, что позволяет оценить трудоёмкость алгоритмов независимо от внешних факторов.

Результаты экспериментов

В данном разделе приводятся результаты экспериментального сравнения Heap Sort и Shell Sort.

Экспериментальные данные

Эксперименты проводились на массивах размером $n = 10, 100, 1000, 10000$ элементов, на четырёх типах входных данных:

- **Sorted** — массив упорядочен по возрастанию
- **Reverse** — массив упорядочен по убыванию (обратный порядок)
- **Random1** — случайные данные (seed = 123)
- **Random2** — случайные данные (seed = 456)

Каждый тест повторялся 5 раз, результаты усредняются.

Heap Sort

n	Параметр	Тип входных данных			
		Sorted	Reverse	Random1	Random2
10	Сравнения	35	41	37	39
	Перемещения	21	30	26	27
100	Сравнения	944	1081	1023	1038
	Перемещения	516	640	587	582
1000	Сравнения	15965	17583	16842	16856
	Перемещения	8316	9708	9090	9066
10000	Сравнения	226682	244460	235277	235329
	Перемещения	116696	131956	124088	124197

Таблица 1: Результаты работы Heap Sort

Shell Sort

n	Параметр	Тип входных данных			
		Sorted	Reverse	Random1	Random2
10	Сравнения	27	22	29	29
	Перемещения	57	44	56	55
100	Сравнения	668	503	793	859
	Перемещения	1266	1006	1343	1416
1000	Сравнения	11716	8006	14866	15062
	Перемещения	20712	16012	23391	23560
10000	Сравнения	172578	120005	266889	262381
	Перемещения	302570	240010	391960	387497

Таблица 2: Результаты работы Shell Sort

Анализ результатов

На основе полученных экспериментальных данных можно сделать следующие наблюдения:

Heap Sort:

- Демонстрирует стабильное поведение, почти независимое от типа входных данных.
- Количество сравнений и перемещений растёт примерно согласно $O(n \log n)$. Например, при увеличении n от 1000 до 10000 (в 10 раз) количество сравнений увеличивается примерно в 14 раз, что близко к $10 \log_2 10 \approx 33$ с коэффициентом реализации.
- На отсортированных и обратно отсортированных данных показывает примерно одинаковые результаты.

Shell Sort:

- Демонстрирует высокую адаптивность к типу входных данных.
- На отсортированных данных значительно быстрее, чем Heap Sort: 172578 сравнений против 226682 для Heap Sort при $n = 10000$.
- На обратно отсортированных данных также показывает улучшенные результаты: 120005 сравнений против 244460 для Heap Sort.
- На случайных данных показывает более высокое значение операций по сравнению с Heap Sort.
- Общая тенденция: меньше сравнений, но больше перемещений, что компенсирует выигрыш в сравнениях.

Структура программы и спецификация функций

Программа состоит из трёх основных файлов:

sort.h

Заголовочный файл, определяющий интерфейс модуля сортировки:

- `void generate_arr(int array[], int size, int type)` — генерирует массив заданного типа и размера
- `int* copy_arr(const int src_arr[], int size)` — создаёт копию массива
- `void heap_sort(int array[], int size)` — сортирует массив методом кучи по убыванию

- `void shell_sort(int array[], int size)` — сортирует массив методом Шелла по убыванию
- `void reset_stats(void)` — обнуляет счётчики операций
- `long long get_heap_comps(void)` — возвращает количество сравнений в Heap Sort
- `long long get_heap_moves(void)` — возвращает количество перемещений в Heap Sort
- `long long get_shell_comps(void)` — возвращает количество сравнений в Shell Sort
- `long long get_shell_moves(void)` — возвращает количество перемещений в Shell Sort

sort.c

Реализация алгоритмов сортировки с подсчётом операций. Используются статические переменные для ведения счётчиков:

- `static long long heap_comps, heap_moves` — счётчики для Heap Sort
- `static long long shell_comps, shell_moves` — счётчики для Shell Sort

Heap Sort реализована в две фазы:

1. Построение минимальной кучи из исходного массива (функция `heapify`)
2. Итеративное извлечение минимума и перестройка кучи

Shell Sort использует последовательность промежутков $n/2, n/4, \dots, 1$ и для каждого промежутка выполняет сортировку вставками.

main.c

Программа тестирования, которая:

1. Определяет размеры тестов: $n = 10, 100, 1000, 10000$
2. Для каждого размера выполняет 5 итераций
3. Для каждой итерации генерирует 4 типа массивов
4. Вызывает оба алгоритма и собирает статистику
5. Усредняет результаты и выводит таблицу

Отладка программы, тестирование функций

Для тестирования и отладки методов сортировки выполнены следующие проверки:

1. Тест 1: Малые массивы ($n = 10$)

- Проверена корректность сортировки по убыванию
- Убедились, что счётчики инициализируются правильно
- Проверили работу на четырёх типах входных данных

2. Тест 2: Средние массивы ($n = 100, 1000$)

- Проверена масштабируемость алгоритмов
- Убедились, что результаты reproducible (повторяемые) при одинаковых seed'ах
- Проверена освобождение памяти (отсутствие memory leaks)

3. Тест 3: Большие массивы ($n = 10000$)

- Проверена корректность работы на больших объёмах данных
- Убедились в отсутствии buffer overflow'ов
- Проверена корректность вычисления среднего значения

Все тесты пройдены успешно. Программа работает стабильно без ошибок и утечек памяти.

Анализ допущенных ошибок

При реализации программы критические ошибки не были обнаружены. Оба алгоритма — Heap Sort и Shell Sort — корректно реализуют соответствующие методы сортировки по убыванию.

Программа успешно прошла тестирование на всех размерах входных массивов ($n = 10, 100, 1000, 10000$) и четырёх типах входных данных без потери корректности или утечек памяти.

Выводы

На основе проведённых экспериментов можно сделать следующие выводы:

1. **Heap Sort** обеспечивает предсказуемую производительность $O(n \log n)$ независимо от типа входных данных, что делает его надёжным выбором при необходимости гарантированного времени выполнения.
2. **Shell Sort** демонстрирует высокую адаптивность к входным данным, достигая значительного преимущества на отсортированных и частично упорядоченных массивах, что объясняется его базированием на сортировке вставками.
3. Для практического применения рекомендуется:
 - использовать Heap Sort, когда требуется гарантированная $O(n \log n)$ производительность;
 - использовать Shell Sort для массивов среднего размера с вероятностью частичной упорядоченности;
 - для очень больших наборов данных рассмотреть использование быстрой сортировки (Quick Sort) или сортировки слиянием (Merge Sort).
4. Подсчёт операций сравнения и перемещения — эффективный метод анализа алгоритмов, позволяющий объективно сравнивать их эффективность независимо от деталей реализации.

Список литературы

- [1] Heap Sort on GeeksforGeeks. <https://www.geeksforgeeks.org/heap-sort/>
Дата обращения: 24 февраля 2026.
- [2] Shell Sort on GeeksforGeeks. <https://www.geeksforgeeks.org/shellsort/>
Дата обращения: 24 февраля 2026.
- [3] Comparison of Sorting Algorithms on Wikipedia.
https://en.wikipedia.org/wiki/Sorting_algorithm Дата обращения: 24 февраля 2026.
- [4] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. Второе издание. — М.: «Вильямс», 2005.